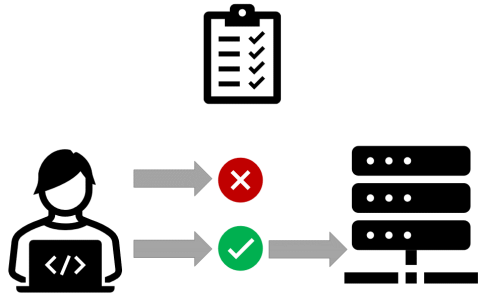
Høgskulen
på Vestlandet

DAT108 - Programmering og webapplikasjoner



F18 - Validering av brukerinput

Lars-Petter Helland
Bergen
16. oktober 2026

Hva så vi på forrige gang?

Brukerinput får man gjerne fra et input-skjema.

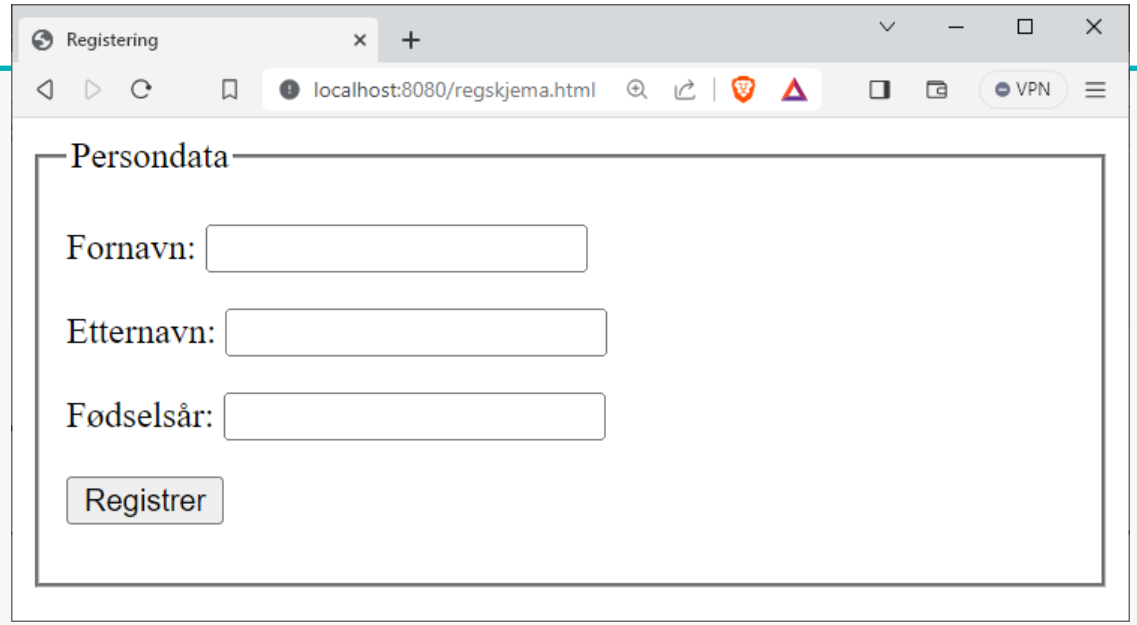
Et slikt skjema angis med en <form>-tag.

Denne inneholder én eller flere <input>-tagger.

Når skjemaet "sendes inn" ved å trykke på submit-knappen, vil verdiene i <input>-taggene overføres med requesten som request-parametre (name=value).

Ved GET vil parameterne bli en del av request-URL-en (og adresselinjen i webleser).

Ved POST vil parameterne sendes med i request-body-en.




```
<form action="registrer" method="post">
  <fieldset>
    <legend>Persondata</legend>
    <p>Fornavn:   <input type="text" name="fornavn"></p>
    <p>Etternavn: <input type="text" name="etternavn"></p>
    <p>Fødselsår: <input type="text" name="aar"></p>
    <p>           <input type="submit" value="Registrer"></p>
  </fieldset>
</form>
```

Hva så vi på forrige gang?

På serveren har man en @Controller-metode som er mappet til "stien" oppgitt i <form>-taggen sin action-attributt og ønsket metode (GET eller POST) oppgitt i method-attributten, f.eks.:

```
@PostMapping("/registrer")
public String mottaRegistrering(Model model,
    @RequestParam String fornavn, ...)
```

Controlleren legger inn data i en model, og spiller deretter over ballen til et view, som genererer en respons tilbake til brukeren, f.eks. en HTML-side.

```
model.addAttribute("p", person);//Til bruk i view
return "kvittering";                //Navn på view
```

```
<form action="registrer" method="post">
  <fieldset>
    <legend>Persondata</legend>
    <p>Fornavn:   <input type="text" name="fornavn"></p>
    <p>Etternavn: <input type="text" name="etternavn"></p>
    <p>Fødselsår: <input type="text" name="aar"></p>
    <p>           <input type="submit" value="Registrer"></p>
  </fieldset>
</form>
```

```
<body>
    <p th:text="|${p.fornavn} ${p.etternavn} er registrert.|"/>
</body>
```

Hva skal vi se på i dag?

- Validering av brukerinput: Hvorfor, hvor, hvordan
- Hvordan kan brukerinput være ugyldig
- Programmatisk validering av enkeltparametre
- Hva gjør vi hvis brukerinput er ugyldig
- Data Binding og @ModelAttribute
- Deklarativ validering og håndtering med Bean Validation
- Regulære uttrykk
- Ufarliggjøring av brukerinput. Escaping.

Dagens lesestoff

Om ulike alternativer i mottak av request-parametre:

- Spring @RequestParam Annotation https://www.baeldung.com/spring-request-param

Om binding av request-parametre til en modell-attributt:

- Spring MVC and the @ModelAttribute Annotation https://www.baeldung.com/spring-mvc-and-the-modelattribute-annotation
- @ModelAttribute https://rwinch.github.io/spring-framework/web/webmvc/mvc-controller/ann-methods/modelattrib-method-args.html

Kort artikkel om hvorfor input-validering er viktig:

- Input validation errors: The root of all evil ... https://www.invicti.com/blog/web-security/input-validation-errors-root-of-all-evil/

Hva er Java Bean Validation:

- Java Bean Validation Basics https://www.baeldung.com/java-validation

Skulle hatt noe om @Valid @ModelAttribute og BindingResult i Spring MVC

Validering av brukerinput: Hvor, hvorfor, hvordan

På klienten

- Hjelp og veiledning slik at bruker får sendt av gårde gyldige data. Gode tilbakemeldinger. Brukervennlighet.
- HTML (deklarativ), JavaScript (programmatisk)

Gjør BÅDE klientside-validering og serverside-validering !!

På tjeneren

- Sikre at applikasjonen kan utføre det den blir bedt om. Sikre mot utilsiktet (og ondsinnet) bruk. Robusthet og sikkerhet.
- Java (programmatisk / deklarativ)



Blacklisting vs. Whitelisting (vs. Sanitazion)

- Blacklisting
  - Å sjekke input mot en liste av ulovlig input.
  - Vanligvis **IKKE** foretrukket. Det kan være lett å glemme noe.
  - (Senere forelesning: Blacklisting anbefales som ett av flere tiltak for økt passordsikkerhet)
- Whitelisting
  - Å liste opp hva som ER lovlig, f.eks. i form av regulære uttrykk eller område av verdier.
  - Vanligvis den **FORETRUKNE** måten. Har da god kontroll på hva som slipper gjennom.
- Sanitazion (rensing)
  - Å fikse på ugyldig input slik at den blir gyldig, f.eks. fjerne ugyldige tegn i en tekst.
  - Dette er **ikke anbefalt å gjøre ved validering** av brukerinput, men handler mer om å ufarliggjøre input ved senere bruk (kommer tilbake til dette i en senere forelesning).

Hvordan kan brukerinput være ugyldig

Eksempel "*Et gyldig fødselsår er 4 siffer og er mellom 1900 og året vi er i nå*"

```
@RequestParam int fodselsaar
```

- parameteren mangler helt i requesten
- parameteren er med, men er tom/blank
- parameteren er med, men har feil type (kan ikke parses til en int)
- parameteren har riktig type, men ugyldig verdi
  - f.eks. for lite eller for stort tall
  - f.eks. feil antall sifre

Vi kan se på hvordan vi sjekker alle disse tingene.

Eksempel - Validere fødselsår

Prosjekt: 📁 f18a-validere-fodselsaar



Oppsummert: Full kontroll / validering av brukerinput

```
@RequestParam int fodselsaar
```

- parameteren mangler helt =>

```
WARN ... MissingServletRequestParameterException:
Required request parameter 'fodselsaar' for method
parameter type int is not present]
```

Mulig løsning: `@RequestParam(required = false)` kan brukes på objekter (f.eks Integer), men ikke primitive typer som int. Objektet blir **null** hvis parameteren mangler.

- parameteren er med, men har feil type =>

```
WARN ... MethodArgumentTypeMismatchException:
Failed to convert value of type 'java.lang.String'
to required type 'int'; For input string: "knut"]
```

Mulig løsning: Kan la parameteren være en **String**, `@RequestParam(required = false) String fodselsaar`, sjekke at det er et heltall programmatisk, og deretter parse.

- parameteren har riktig type, men ugyldig verdi, f.eks. for lite eller for stort tall eller feil antall sifre

Mulig løsning: Kan validere i f.eks. en hjelpemetode/-klasse (programmatisk).

Oppsummert: Validator-hjelpeklasse og JUnit

- Det aller første som bør gjøres når applikasjonen mottar en forespørsel er å sjekke at alt er gyldig (*en. valid*), f.eks. request-parametre.

- Spring-controlleren bør ikke gjøre dette selv (den har nok å gjøre), men bruke en hjelpeklasse.

- En slik validator-klasse kan f.eks. ha metoder som returnerer true/false ved sjekk om en verdi er gyldig, eller ha metoder som returnerer feilkode/-melding som sier mer om hva som var feil.

- For å sikre at validator-metodene er korrekte, bør de testes med JUnit

Litt recap Thymeleaf

Attributter (som er lagt til i controlleren) hentes ved å bruke attributt-navnet direkte:

```
<p th:text="${person}" ></p>
```

Objektegenskaper hentes ved å bruke . (NB! Det er getter-metoden som kalles):

```
<p th:text="${person.fornavn}" ></p>
```

Vi kan skjøte sammen ting med å bruke |...|:

```
<p th:text="|Hei, ${person.fornavn}|" ></p>
```

Request-parametre kan hentes fra param-objektet:

```
<p th:text="${param.fodselsaar}" ></p>
```

Hvis vi ønsker verdien rett inn i html-en, og ikke i en tag:

```
<th:block th:text="${melding}"></th:block>
```

En nyttig operator er ? : (Betinget-operatoren):

```
<p th:text="${gyldig ? "Gyldig" : "Ikke gyldig"}" ></p>
```

Data Binding

Data Binding in Spring MVC with Example

https://www.geeksforgeeks.org/java/data-binding-in-spring-mvc-with-example/

Data Binding in Spring MVC is a mechanism that automatically maps HTTP request parameters (form data, query parameters, etc.) directly into Java objects, making it easier to capture and process user input on the server side. It is commonly used in applications like registration forms, login pages, and Google Forms where user input needs to be stored and processed efficiently.

- *It automatically binds form fields to Java object properties.*
- *It removes the need to manually use request.getParameter() for each input.*
- *It supports simple types, objects, and even collections for complex forms.*

When a user submits a form in Spring MVC, the framework automatically converts the incoming request data into a Java object using data binding. This removes the need for manual parsing and makes request handling clean and efficient.

Data Binding og @ModelAttribute

https://rwinch.github.io/spring-framework/web/webmvc/mvc-controller/ann-methods/modelattrib-method-args.html (Spring ref.):

> *You can use the @ModelAttribute annotation on a method argument to access an attribute from the model or have it instantiated if not present.* ***The model attribute is also overlain with values from HTTP request parameters whose names match to field names****. This is referred to as data binding, and it saves you from having to deal with parsing and converting individual query parameters and form fields.*

Vi skal se hvordan bruk av Data Binding og @ModelAttribute kan fange opp et helt skjema av request-parametre i ett enkelt objekt (f.eks. et Person-objekt).

Eksempel - Data binding og @ModelAttribute

Prosjekt: 📁 f18b-validere-person



Ting å legge merke til

`Person person` vil prøve å binde request-parametre til egenskaper i Person

`@ModelAttribute Person person` vil i tillegg legge det til som en modell-attributt

- Hvis (men ikke vist i eksempelet) dataene i skjemaet ikke er akkurat det samme som i dataklassen (Person), er det vanlig å lage en egen klasse (f.eks. `PersonSkjema`) med akkurat de dataene som er i skjemaet. Dette blir ofte kalt et **Form Object**.
- *Men kjært barn har mange navn: Command Object (offisielt), Form Object/Bean, Data Transfer Object (DTO), ..*


Pause

Deklarativ validering og håndtering

Vi skal nå se hvordan vi kan forenkle valideringen ved å annotere i f.eks. Person-klassen.

For å kunne gjøre dette trenger vi denne i pom.xml:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Eksempel - Validering av person med Bean Validation

Prosjekt: 📂 f18b-validere-person

Fortsetter med samme prosjekt som i sted



Oppsummert: Deklarativ validering og håndtering

`@ModelAttribute`

Gir oss mulighet til å mappe et helt input-skjema til ett objekt og legge det til i modellen.

`@NotNull, @Pattern, @Size, @Min, ...`

Setter valideringsregler i objektet det mappes til!

`@Valid @ModelAttribute + BindingResult`

Angir at vi sjekker valideringsreglene og legger resultet i et BindingResult.

PS! Det er mer man kan gjøre med BindingResult enn det som ble vist i eksemplet.

Hva vi gjør ved ugyldig input er litt opp til oss, men vi kan f.eks. la bruker få et nytt forsøk. Da kan vi legge inn eksisterende verdier fra @ModelAttribute og info fra BindingResult i viewet (som genererer HTML for nytt forsøk).

Oppsummert: JUnit

Det er også mulig å teste at valideringsreglene vi har satt opp fungerer som ønsket:

- Vi oppretter et Validator-objekt
- Vi bruker dette til å validere diverse eksempler på objekter, og sjekker at valideringen gir de forventede feil (violations).

```java
public class PersonValideringTest {

    private Validator validator;

    private Person testPerson;

    @BeforeEach
    void setUp() {
        validator = Validation.buildDefaultValidatorFactory().getValidator();

        testPerson = new Person();
        testPerson.setNavn("Arne");
        testPerson.setMobil("12345678");
        testPerson.setHoydecm(180);
        testPerson.setFdato(LocalDate.of(1990, 5, 30));
    }
```

```java
@Test
void navnErObligatorisk() {
    testPerson.setNavn(null);
    sjekkAtPersonErUgyldigMedDenneEneFeilen("Navn er obligatorisk");
}
```

Oppsummert: Litt mer Thymeleaf

Hvis vi ønsker å kunne angi feilmeldinger på en side, kan vi sette av plass til dette, f.eks. slik:

```
<p style="color:red;" th:text="${feilmelding}"></p>
```

Hvis vi ønsker at brukerens forrige inntasting skal "henge igjen" ved nye forsøk, kan dette gjøres ved å sette value på input-elementet:

```
<input type="text" name="navn" th:value="${person.navn}">
```

Da blir altså input-feltet forhåndsutfylt med verdien til egenskapen navn i modell-attributten person, som enten ble lagt til i controlleren manuelt med

```
model.addAttribute(«person», person);
```

eller med

```
@ModelAttribute Person person
```

Thymeleaf-trick for å unngå at programmet kræsjer (NullPointerException) hvis `person` er `null` og vi prøver å gjøre `person.navn`. Vi setter inn et `?` :

```
<input type="text" name="navn" th:value="${person?.navn}">
```

Denne (?.) kalles Safe Navigation Operator, og gjør en implisitt null-sjekk.

Regulære uttrykk

- Ikke pensum å kunne dette i detalj, men dere skal vite hva det er og hvordan det kan brukes i validering.
- En fin side for å lære om regulære uttrykk er https://regexone.com/

Ufarliggjøring av brukerinput. Escaping

Det kan være at gyldig brukerinput ikke egner seg direkte til f.eks. å plassere i en HTML-side eller å lagre i en database. F.eks. vil:

Navn `<h1>BØ</h1>`

innsatt i en HTML-side føre til at det settes inn en <h1>-tag, eller:

Navn `'; DROP TABLE forfatter;`

innsatt i en SQL-(select)setning føre til at tabellen forfatter slettes.

Vi skal nå se at vi kan **ufarliggjøre brukerinput** for visning i en HTML-side ved å bruke JSTL-taggen

```
<c:out ... />
```

Den vil erstatte «farlige» HTML-tegn med koder. Kalles også **escaping**.

Ufarliggjøring av brukerinput. Escaping

Vi kan prøve dette på et av våre eksempler



Hva har vi sett på i dag

- Validering av brukerinput: Hvorfor, hvor, hvordan
- Hvordan kan brukerinput være ugyldig
- Programmatisk validering av enkeltparametre
- Hva gjør vi hvis brukerinput er ugyldig
- Data Binding og @ModelAttribute
- Deklarativ validering og håndtering med Bean Validation
- Regulære uttrykk
- Ufarliggjøring av brukerinput. Escaping.

Husk: "All brukerinput MÅ valideres på server!"

Videre

- Oblig4 ligger ute ... NB! Frist søndag 25. okt.

- Mandag
  - Sesjoner og innlogging (inkl. PRG)
  - Sentralt i Oblig4

- Fredag
  - Lagdeling i Spring-applikasjoner, Spring Dependency Injection (DI), @Service, @Autowired, Spring Data JPA, repositories, ...

    (Sagt på en annen måte. Koble oss opp til database!)
  - Trengs ikke i Oblig4, men viktig i Oblig5